

第 9 章

使用 ssh 服务管理远程主机

本章讲解了如下内容：

- 配置网络服务；
- 远程控制服务；
- 不间断会话服务。

本章讲解了如何使用 `nmtui` 命令配置网络参数，以及通过 `nmcli` 命令查看网络信息并管理网络会话服务，从而让您能够在不同工作场景中快速地切换网络运行参数；还讲解了如何手工绑定 `mode6` 模式双网卡，实现网络的负载均衡。。

本章还深入介绍了 SSH 协议与 `sshd` 服务程序的理论知识、Linux 系统的远程管理方法以及在系统中配置服务程序的方法，并采用实验的形式演示了使用基于密码验证的 `sshd` 服务程序进行远程登录，以及使用 `screen` 服务程序远程管理 Linux 系统的不间断会话等技术。

当读者掌握了本章的内容之后，也就完全具备了对 Linux 系统进行配置管理的知识。而且后续章节中将陆续引入大量实用服务的配置内容，读者将用到本章学习的知识进行配置，这样一方面可以让读者对生产环境中用到的大多数热门服务程序有一个广泛且深入的认识，另一方面也可以掌握相应的配置方法。

9.1 配置网络服务

9.1.1 配置网络参数

截至目前，大家已经完全可以利用当前所学的知识来管理 Linux 系统了。当然，大家的水平完全可以更进一步，当有朝一日登顶技术巅峰时，您一定会感谢现在正在努力学习的您。

我们接下来将学习如何在 Linux 系统上配置服务。但是在此之前，必须先保证主机之间能够顺畅地通信。如果网络不通，即便服务部署得再正确用户也无法顺利访问，所以，配置网络并确保网络的连通性是学习部署 Linux 服务之前的最后一个重要知识点。

在 4.1.3 小节讲解了如何使用 Vim 文本编辑器来配置网络参数，其实，在 RHEL 7 系统中有至少 5 种网络的配置方法，刘遄老师尽量在本书中为大家逐一演示。这里教给大家的是使

用 `nmtui` 命令来配置网络，其具体的配置步骤如图 9-1 至图 9-8 所示。当遇到不容易理解的内容时，我们会额外进行解释说明。

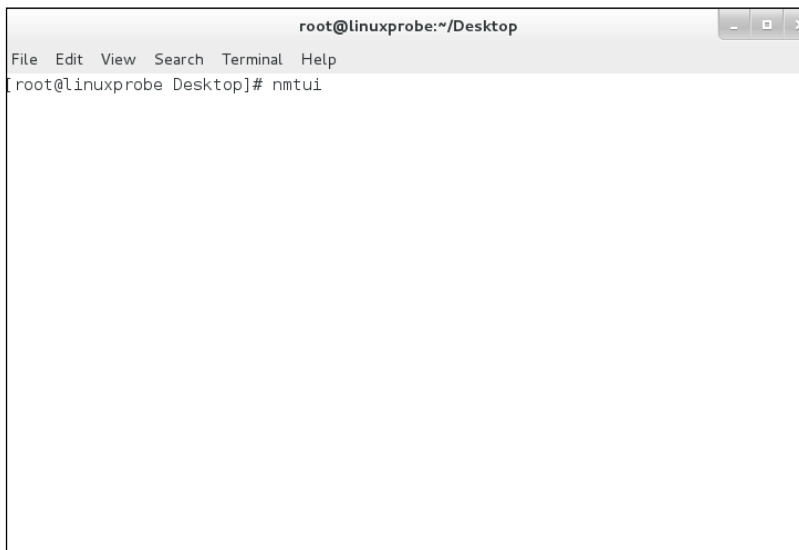


图 9-1 执行 `nmtui` 命令运行网络配置工具

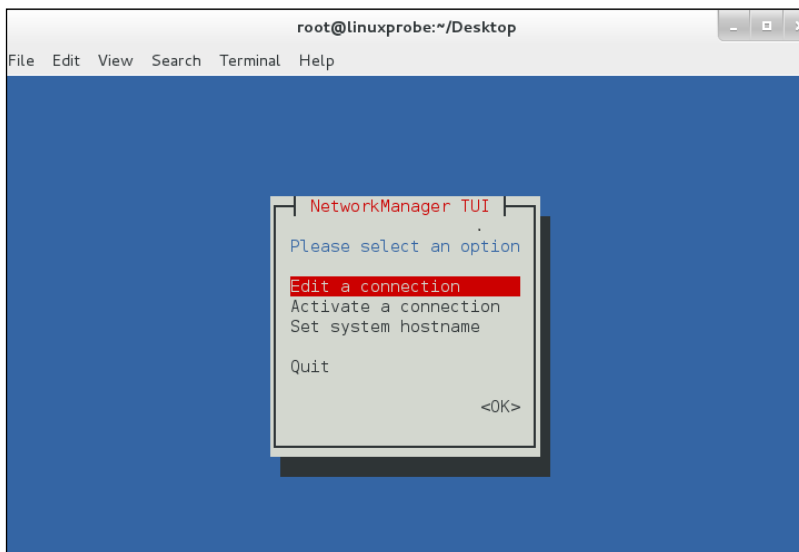


图 9-2 选中 `Edit a connection` 并按下回车键

在 RHEL 5、RHEL 6 系统及其他大多数早期的 Linux 系统中，网卡的名称一直都是 `eth0`、`eth1`、`eth2`、……，但在 RHEL 7 中则变成了类似于 `eno16777736` 这样的名字。不过除了网卡的名称发生变化之外，其他几乎一切照旧，因此这里演示的网络配置实验完全可以适用于各种版本的 Linux 系统。

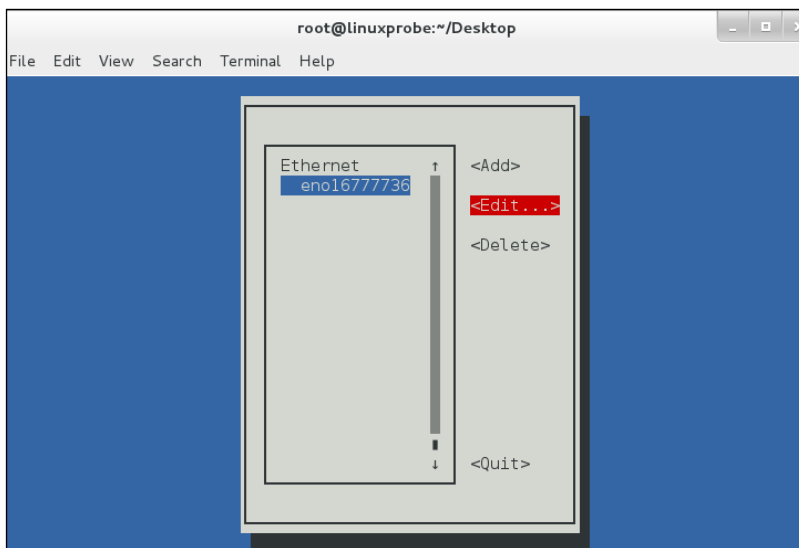


图 9-3 选中要编辑的网卡名称，然后按下 Edit（编辑）按钮

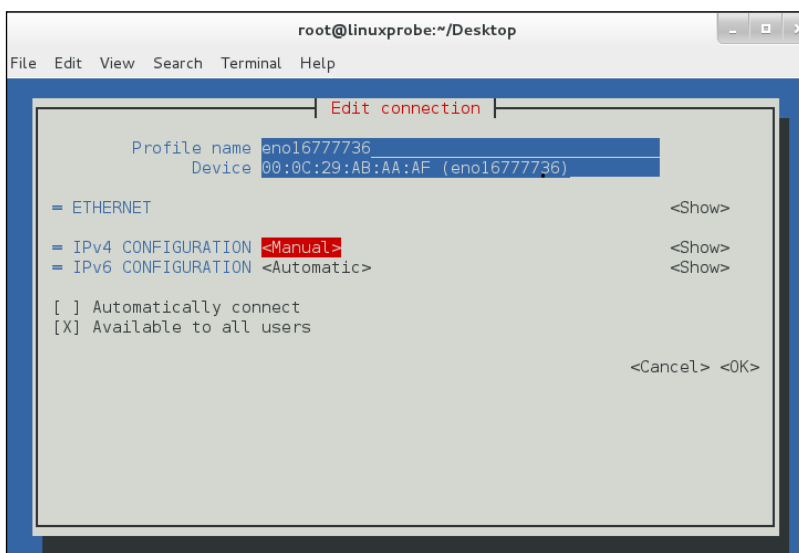


图 9-4 把网络 IPv4 的配置方式改成 Manual（手动）

注：

再多提一句，我们的这本《Linux 就该这么学》不仅学习门槛低、简单易懂，而且还有一个潜在的优势——书中所有的服务器主机 IP 地址均为 192.168.10.10，而客户端主机均为 192.168.10.20 及 192.168.10.30。这样的好处就是，在后面部署 Linux 服务的时候，不用每次都要考虑 IP 地址变化的问题，从而可以心无旁骛地关注配置细节。

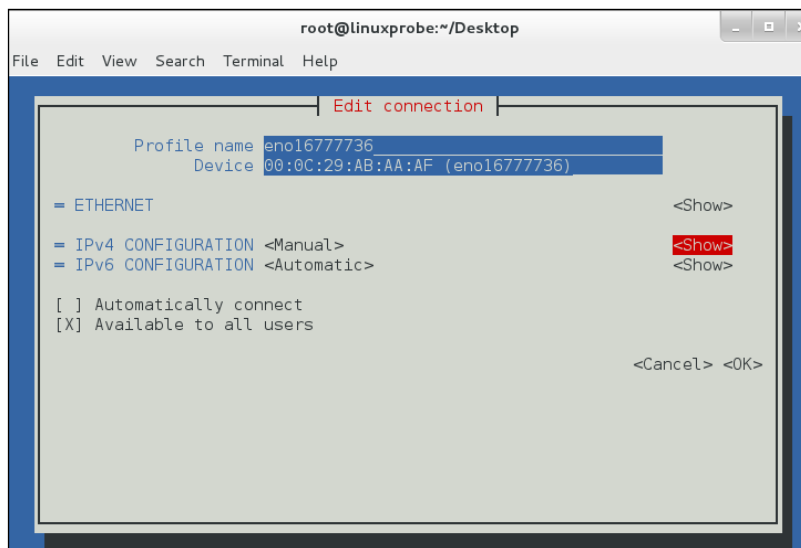


图 9-5 按下 Show（显示）按钮，显示信息配置框

现在，在服务器主机的网络配置信息中填写 IP 地址 192.168.10.10/24。

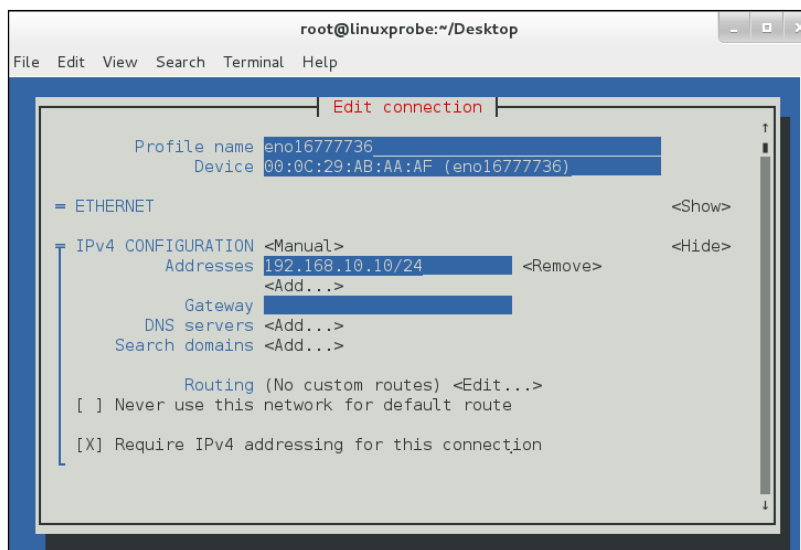


图 9-6 填写 IP 地址

至此，在 Linux 系统中配置网络的步骤就结束了。

刘遄老师在培训时经常会发现，很多学员在安装 RHEL 7 系统时默认没有激活网卡。如果各位读者有同样的情况也不用担心，只需使用 Vim 编辑器将网卡配置文件中的 ONBOOT 参数修改成 yes，这样在系统重启后网卡就被激活了。

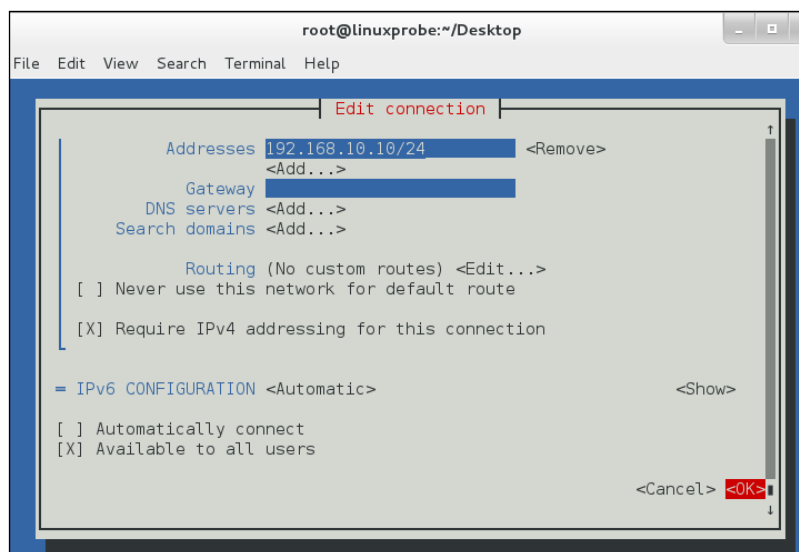


图 9-7 单击 OK 按钮保存配置

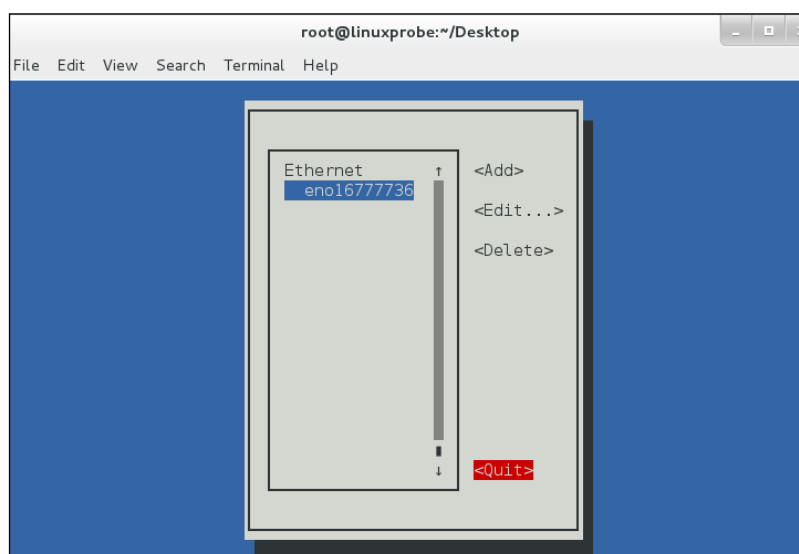


图 9-8 单击 Quit 按钮退出

```
[root@linuxprobe ~]# vim /etc/sysconfig/network-scripts/ifcfg-eno1677736
TYPE=Ethernet
BOOTPROTO=none
DEFROUTE=yes
IPV4_FAILURE_FATAL=no
IPV6INIT=yes
IPV6_AUTOCONF=yes
IPV6_DEFROUTE=yes
IPV6_FAILURE_FATAL=no
NAME=eno1677736
UUID=ec77579b-2ced-481f-9c09-f562b321e268
ONBOOT=yes
IPADDR0=192.168.10.10
```

```
HWADDR=00:0C:29:C4:A4:09
PREFIX0=24
IPV6_PEERDNS=yes
IPV6_PEERROUTES=yes
```

当修改完 Linux 系统中的服务配置文件后，并不会对服务程序立即产生效果。要想让服务程序获取到最新的配置文件，需要手动重启相应的服务，之后就可以看到网络畅通了：

```
[root@linuxprobe ~]# systemctl restart network
[root@linuxprobe ~]# ping -c 4 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=64 time=0.056 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=64 time=0.099 ms
64 bytes from 192.168.10.10: icmp_seq=3 ttl=64 time=0.095 ms
64 bytes from 192.168.10.10: icmp_seq=4 ttl=64 time=0.095 ms

--- 192.168.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2999ms
rtt min/avg/max/mdev = 0.056/0.086/0.099/0.018 ms
```

9.1.2 创建网络会话

RHEL 和 CentOS 系统默认使用 NetworkManager 来提供网络服务，这是一种动态管理网络配置的守护进程，能够让网络设备保持连接状态。可以使用 nmcli 命令来管理 Network Manager 服务。nmcli 是一款基于命令行的网络配置工具，功能丰富，参数众多。它可以轻松地查看网络信息或网络状态：

```
[root@linuxprobe ~]# nmcli connection show
NAME UUID TYPE DEVICE
eno16777736 ec77579b-2ced-481f-9c09-f562b321e268 802-3-ethernet eno16777736
[root@linuxprobe ~]# nmcli con show eno16777736
connection.id: eno16777736
connection.uuid: ec77579b-2ced-481f-9c09-f562b321e268
connection.interface-name: --
connection.type: 802-3-ethernet
connection.autoconnect: yes
connection.timestamp: 1487348994
connection.read-only: no
connection.permissions:
connection.zone: --
connection.master: --
connection.slave-type: --
connection.secondaries:
connection.gateway-ping-timeout: 0
.....省略部分输出信息.....
```

另外，RHEL7 系统支持网络会话功能，允许用户在多个配置文件中快速切换（非常类似于 firewallld 防火墙服务中的区域技术）。如果我们在公司网络中使用笔记本电脑时需要手动指定网络的 IP 地址，而回到家中则是使用 DHCP 自动分配 IP 地址。这就需要麻烦地频繁修改 IP 地址，但是使用了网络会话功能后一切就简单多了——只需在不同的使用环境中激活相应的网络会话，就可以实现网络配置信息的自动切换了。

可以使用 `nmcli` 命令并按照“`connection add con-name type ifname`”的格式来创建网络会话。假设将公司网络中的网络会话称之为 `company`，将家庭网络中的网络会话称之为 `house`，现在依次创建各自的网络会话。

使用 `con-name` 参数指定公司所使用的网络会话名称 `company`，然后依次用 `ifname` 参数指定本机的网卡名称（千万要以实际环境为准，不要照抄书上的 `eno16777736`），用 `autoconnect no` 参数设置该网络会话默认不被自动激活，以及用 `ip4` 及 `gw4` 参数手动指定网络的 IP 地址：

```
[root@linuxprobe ~]# nmcli connection add con-name company ifname eno16777736
autoconnect no type ethernet ip4 192.168.10.10/24 gw4 192.168.10.1
Connection 'company' (86c71220-0057-419e-b615-38f4014cfdee) successfully added.
```

使用 `con-name` 参数指定家庭所使用的网络会话名称 `house`。因为我们想从外部 DHCP 服务器自动获得 IP 地址，因此这里不需要进行手动指定。

```
[root@linuxprobe ~]# nmcli connection add con-name house type ethernet ifname
eno16777736
Connection 'house' (44acf0a7-07e2-40b4-94ba-69ea973090fb) successfully added.
```

在成功创建网络会话后，可以使用 `nmcli` 命令查看创建的所有网络会话：

```
[root@linuxprobe ~]# nmcli connection show
NAME UUID TYPE DEVICE
house 44acf0a7-07e2-40b4-94ba-69ea973090fb 802-3-ethernet --
company 86c71220-0057-419e-b615-38f4014cfdee 802-3-ethernet --
eno16777736 ec77579b-2ced-481f-9c09-f562b321e268 802-3-ethernet eno16777736
```

使用 `nmcli` 命令配置过的网络会话是永久生效的，这样当我们下班回家后，顺手启用 `house` 网络会话，网卡就能自动通过 DHCP 获取到 IP 地址了。

```
[root@linuxprobe ~]# nmcli connection up house
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/
ActiveConnection/2)
[root@linuxprobe ~]# ifconfig
eno1677773628: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.100.128 netmask 255.255.255.0 broadcast 192.168.100.255
inet6 fe80::20c:29ff:fec4:a409 prefixlen 64 scopeid 0x20<link>
ether 00:0c:29:c4:a4:09 txqueuelen 1000 (Ethernet)
RX packets 42 bytes 4198 (4.0 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 75 bytes 10441 (10.1 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 0 (Local Loopback)
RX packets 518 bytes 44080 (43.0 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 518 bytes 44080 (43.0 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

如果大家使用的是虚拟机，请把虚拟机系统的网卡（网络适配器）切换成桥接模式，如

图 9-9 所示。然后重启虚拟机系统即可。

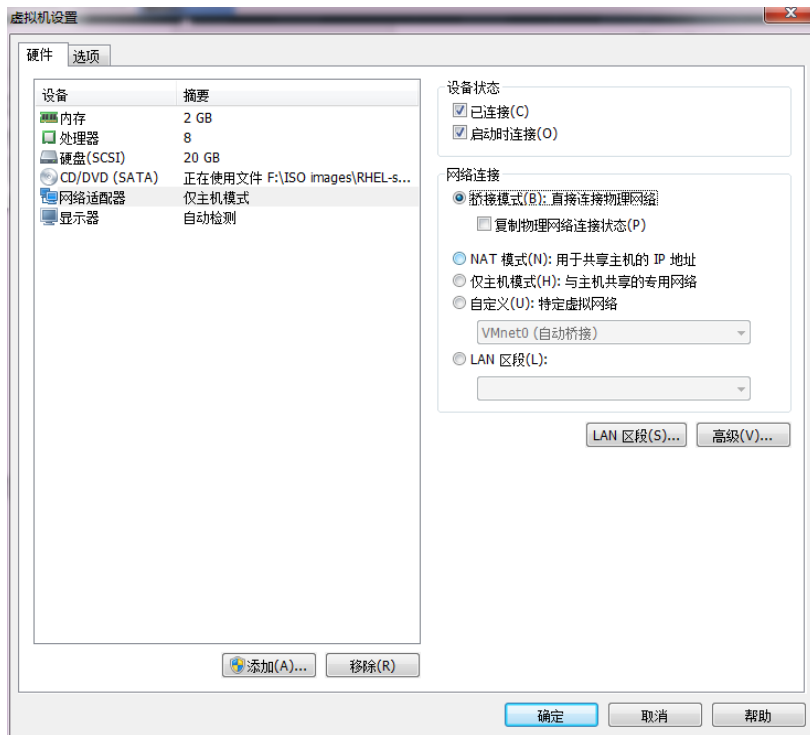


图 9-9 设置虚拟机网卡的模式

9.1.3 绑定两块网卡

一般来讲，生产环境必须提供 7×24 小时的网络传输服务。借助于网卡绑定技术，不仅可以提高网络传输速度，更重要的是，还可以确保在其中一块网卡出现故障时，依然可以正常提供网络服务。假设我们对两块网卡实施了绑定技术，这样在正常工作中它们会共同传输数据，使得网络传输的速度变得更快；而且即使有一块网卡突然出现了故障，另外一块网卡便会立即自动顶替上去，保证数据传输不会中断。

下面我们来看一下如何绑定网卡。

第 1 步：在虚拟机系统中再添加一块网卡设备，请确保两块网卡都处在同一个网络连接中（即网卡模式相同），如图 9-10 和图 9-11 所示。处于相同模式的网卡设备才可以进行网卡绑定，否则这两块网卡无法互相传送数据。

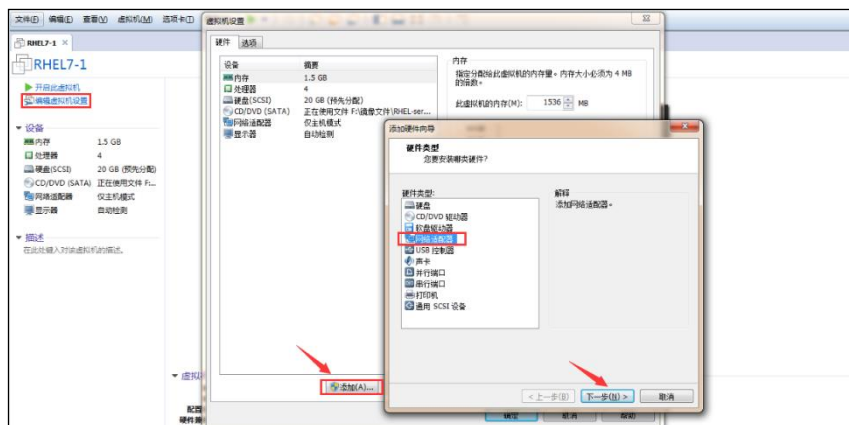


图 9-10 在虚拟机中再添加一块网卡设备

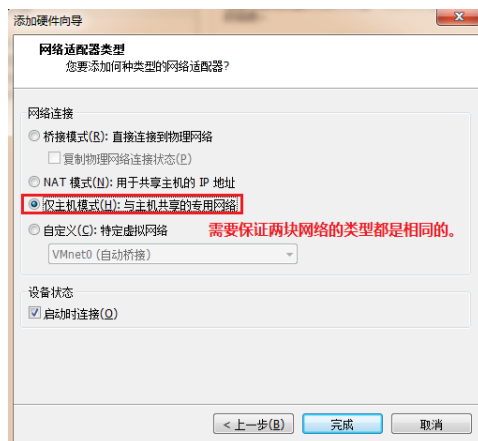


图 9-11 确保两块网卡处在同一个网络连接中（即网卡模式相同）

第 2 步：使用 Vim 文本编辑器来配置网卡设备的绑定参数。网卡绑定的理论知识类似于前面学习的 RAID 硬盘组，我们需要对参与绑定的网卡设备逐个进行“初始设置”。需要注意的是，这些原本独立的网卡设备此时需要被配置成为一块“从属”网卡，服务于“主”网卡，不应该再有自己的 IP 地址等信息。在进行了初始设置之后，它们就可以支持网卡绑定。

```
[root@linuxprobe ~]# vim /etc/sysconfig/network-scripts/ifcfg-eno1677736
TYPE=Ethernet
BOOTPROTO=none
ONBOOT=yes
USERCTL=no
DEVICE=eno1677736
MASTER=bond0
SLAVE=yes
[root@linuxprobe ~]# vim /etc/sysconfig/network-scripts/ifcfg-eno33554968
TYPE=Ethernet
BOOTPROTO=none
ONBOOT=yes
USERCTL=no
DEVICE=eno33554968
MASTER=bond0
```

```
SLAVE=yes
```

还需要将绑定后的设备命名为 `bond0` 并把 IP 地址等信息填写进去, 这样当用户访问相应服务的时候, 实际上就是由这两块网卡设备在共同提供服务。

```
[root@linuxprobe ~]# vim /etc/sysconfig/network-scripts/ifcfg-bond0
TYPE=Ethernet
BOOTPROTO=none
ONBOOT=yes
USERCTL=no
DEVICE=bond0
IPADDR=192.168.10.10
PREFIX=24
DNS=192.168.10.1
NM_CONTROLLED=no
```

第 3 步:让 Linux 内核支持网卡绑定驱动。常见的网卡绑定驱动有三种模式——`mode0`、`mode1` 和 `mode6`。下面以绑定两块网卡为例, 讲解使用的情景。

- **mode0** (平衡负载模式): 平时两块网卡均工作, 且自动备援, 但需要在与服务器本地网卡相连的交换机设备上端口聚合来支持绑定技术。
- **mode1** (自动备援模式): 平时只有一块网卡工作, 在它故障后自动替换为另外的网卡。
- **mode6** (平衡负载模式): 平时两块网卡均工作, 且自动备援, 无须交换机设备提供辅助支持。

比如有一台用于提供 NFS 或者 samba 服务的文件服务器, 它所能提供的最大网络传输速度为 100Mbit/s, 但是访问该服务器的用户数量特别多, 那么它的访问压力一定很大。在生产环境中, 网络的可靠性是极为重要的, 而且网络的传输速度也必须得以保证。针对这样的情况, 比较好的选择就是 `mode6` 网卡绑定驱动模式了。因为 `mode6` 能够让两块网卡同时一起工作, 当其中一块网卡出现故障后能自动备援, 且无需交换机设备支援, 从而提供了可靠的网络传输保障。

下面使用 Vim 文本编辑器创建一个用于网卡绑定的驱动文件, 使得绑定后的 `bond0` 网卡设备能够支持绑定技术 (`bonding`); 同时定义网卡以 `mode6` 模式进行绑定, 且出现故障时自动切换的时间为 100 毫秒。

```
[root@linuxprobe ~]# vim /etc/modprobe.d/bond.conf
alias bond0 bonding
options bond0 miimon=100 mode=6
```

第 4 步:重启网络服务后网卡绑定操作即可成功。正常情况下只有 `bond0` 网卡设备才会有 IP 地址等信息:

```
[root@linuxprobe ~]# systemctl restart network
[root@linuxprobe ~]# ifconfig
bond0: flags=5187<UP,BROADCAST,RUNNING,MASTER,MULTICAST> mtu 1500
inet 192.168.10.10 netmask 255.255.255.0 broadcast 192.168.10.255
inet6 fe80::20c:29ff:fe9c:637d prefixlen 64 scopeid 0x20<link>
ether 00:0c:29:9c:63:7d txqueuelen 0 (Ethernet)
RX packets 700 bytes 82899 (80.9 KiB)
```

```
RX errors 0 dropped 6 overruns 0 frame 0
TX packets 588 bytes 40260 (39.3 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eno16777736: flags=6211<UP,BROADCAST,RUNNING,SLAVE,MULTICAST> mtu 1500
ether 00:0c:29:9c:63:73 txqueuelen 1000 (Ethernet)
RX packets 347 bytes 40112 (39.1 KiB)
RX errors 0 dropped 6 overruns 0 frame 0
TX packets 263 bytes 20682 (20.1 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eno33554968: flags=6211<UP,BROADCAST,RUNNING,SLAVE,MULTICAST> mtu 1500
ether 00:0c:29:9c:63:7d txqueuelen 1000 (Ethernet)
RX packets 353 bytes 42787 (41.7 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 325 bytes 19578 (19.1 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

可以在本地主机执行 `ping 192.168.10.10` 命令检查网络的连通性。为了检验网卡绑定技术的自动备援功能，我们突然在虚拟机硬件配置中随机移除一块网卡设备，可以非常清晰地看到网卡切换的过程（一般只有 1 个数据丢包）。然后另外一块网卡会继续为用户提供服务。

```
[root@linuxprobe ~]# ping 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=64 time=0.109 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=64 time=0.102 ms
64 bytes from 192.168.10.10: icmp_seq=3 ttl=64 time=0.066 ms
ping: sendmsg: Network is unreachable
64 bytes from 192.168.10.10: icmp_seq=5 ttl=64 time=0.065 ms
64 bytes from 192.168.10.10: icmp_seq=6 ttl=64 time=0.048 ms
64 bytes from 192.168.10.10: icmp_seq=7 ttl=64 time=0.042 ms
64 bytes from 192.168.10.10: icmp_seq=8 ttl=64 time=0.079 ms
^C
--- 192.168.10.10 ping statistics ---
8 packets transmitted, 7 received, 12% packet loss, time 7006ms
rtt min/avg/max/mdev = 0.042/0.073/0.109/0.023 ms
```

9.2 远程控制服务

9.2.1 配置 sshd 服务

SSH（Secure Shell）是一种能够以安全的方式提供远程登录的协议，也是目前远程管理 Linux 系统的首选方式。在此之前，一般使用 FTP 或 Telnet 来进行远程登录。但是因为它们以明文的形式在网络中传输账户密码和数据信息，因此很不安全，很容易受到黑客发起的中间人攻击，这轻则篡改传输的数据信息，重则直接抓取服务器的账户密码。

想要使用 SSH 协议来远程管理 Linux 系统，则需要部署配置 sshd 服务程序。sshd 是基于 SSH 协议开发的一款远程管理服务程序，不仅使用起来方便快捷，而且能够提供两种安全验证的方法：

- 基于口令的验证——用账户和密码来验证登录；

- 基于密钥的验证——需要在本地生成密钥对，然后把密钥对中的公钥上传至服务器，并与服务器中的公钥进行比较；该方式相较来说更安全。

前文曾多次强调“Linux 系统中的一切都是文件”，因此在 Linux 系统中修改服务程序的运行参数，实际上就是在修改程序配置文件的过程。sshd 服务的配置信息保存在/etc/ssh/sshd_config 文件中。运维人员一般会把保存着最主要配置信息的文件称为主配置文件，而配置文件中有许多以井号开头的注释行，要想让这些配置参数生效，需要在修改参数后再去掉前面的井号。sshd 服务配置文件中包含的重要参数如表 9-1 所示。

表 9-1 sshd 服务配置文件中包含的参数以及作用

参数	作用
Port 22	默认的 sshd 服务端口
ListenAddress 0.0.0.0	设定 sshd 服务器监听的 IP 地址
Protocol 2	SSH 协议的版本号
HostKey /etc/ssh/ssh_host_key	SSH 协议版本为 1 时，DES 私钥存放的位置
HostKey /etc/ssh/ssh_host_rsa_key	SSH 协议版本为 2 时，RSA 私钥存放的位置
HostKey /etc/ssh/ssh_host_dsa_key	SSH 协议版本为 2 时，DSA 私钥存放的位置
PermitRootLogin yes	设定是否允许 root 管理员直接登录
StrictModes yes	当远程用户的私钥改变时直接拒绝连接
MaxAuthTries 6	最大密码尝试次数
MaxSessions 10	最大终端数
PasswordAuthentication yes	是否允许密码验证
PermitEmptyPasswords no	是否允许空密码登录（很不安全）

在 RHEL 7 系统中，已经默认安装并启用了 sshd 服务程序。接下来使用 ssh 命令进行远程连接，其格式为“ssh [参数] 主机 IP 地址”。要退出登录则执行 exit 命令。

```
[root@linuxprobe ~]# ssh 192.168.10.10
The authenticity of host '192.168.10.20 (192.168.10.10)' can't be established.
ECDSA key fingerprint is 4f:a7:91:9e:8d:6f:b9:48:02:32:61:95:48:ed:1e:3f.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.10.10' (ECDSA) to the list of known hosts.
root@192.168.10.20's password:此处输入远程主机 root 管理员的密码
Last login: Wed Apr 15 15:54:21 2017 from 192.168.10.10
[root@linuxprobe ~]#
[root@linuxprobe ~]# exit
logout
Connection to 192.168.10.10 closed.
```

如果禁止以 root 管理员的身份远程登录到服务器，则可以大大降低被黑客暴力破解密码的几率。下面进行相应配置。首先使用 Vim 文本编辑器打开 sshd 服务的主配置文件，然后把第 48 行#PermitRootLogin yes 参数前的井号（#）去掉，并把参数值 yes 改成 no，这样就不再允许 root 管理员远程登录了。记得最后保存文件并退出。

```
[root@linuxprobe ~]# vim /etc/ssh/sshd_config
.....省略部分输出信息.....
46
47 #LoginGraceTime 2m
```

```
48 PermitRootLogin no
49 #StrictModes yes
50 #MaxAuthTries 6
51 #MaxSessions 10
52
.....省略部分输出信息.....
```

再次提醒的是，一般的服务程序并不会在配置文件修改之后立即获得最新的参数。如果想让新配置文件生效，则需要手动重启相应的服务程序。最好也将这个服务程序加入到开机启动项中，这样系统在下次启动时，该服务程序便会自动运行，继续为用户提供服务。

```
[root@linuxprobe ~]# systemctl restart sshd
[root@linuxprobe ~]# systemctl enable sshd
```

这样一来，当 root 管理员再来尝试访问 sshd 服务程序时，系统会提示不可访问的错误信息。虽然 sshd 服务程序的参数相对比较简单，但这就是在 Linux 系统中配置服务程序的正确方法。大家要做的是举一反三、活学活用，这样即便以后遇到了陌生的服务，也一样可以搞定了。

```
[root@linuxprobe ~]# ssh 192.168.10.10
root@192.168.10.10's password:此处输入远程主机 root 管理员的密码
Permission denied, please try again.
```

9.2.2 安全密钥验证

加密是对信息进行编码和解码的技术，它通过一定的算法（密钥）将原本可以直接阅读的明文信息转换成密文形式。密钥即是密文的钥匙，有私钥和公钥之分。在传输数据时，如果担心被他人监听或截获，就可以在传输前先使用公钥对数据加密处理，然后再行传送。这样，只有掌握私钥的用户才能解密这段数据，除此之外的其他人即便截获了数据，一般也很难将其破译为明文信息。

一言以蔽之，在生产环境中使用密码进行口令验证终归存在着被暴力破解或嗅探截获的风险。如果正确配置了密钥验证方式，那么 sshd 服务程序将更加安全。我们下面进行具体的配置，其步骤如下。

第 1 步：在客户端主机中生成“密钥对”。

```
[root@linuxprobe ~]# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):按回车键或设置密钥的存储路径
Created directory '/root/.ssh'.
Enter passphrase (empty for no passphrase): 直接按回车键或设置密钥的密码
Enter same passphrase again: 再次按回车键或设置密钥的密码
Your identification has been saved in /root/.ssh/id_rsa.
Your public key has been saved in /root/.ssh/id_rsa.pub.
The key fingerprint is:
40:32:48:18:e4:ac:c0:c3:cl:ba:7c:6c:3a:a8:b5:22 root@linuxprobe.com
The key's randomart image is:
+--[ RSA 2048 ]-----+
|+*..o .              |
|*.o  +               |
|o*  .                |
```

```
|+ .      .      |
|o..      S      |
|.. +      |
|. =      |
|E+ .      |
|+.o      |
+-----+
```

第 2 步：把客户端主机中生成的公钥文件传送至远程主机：

```
[root@linuxprobe ~]# ssh-copy-id 192.168.10.10
The authenticity of host '192.168.10.20 (192.168.10.10)' can't be established.
ECDSA key fingerprint is 4f:a7:91:9e:8d:6f:b9:48:02:32:61:95:48:ed:1e:3f.
Are you sure you want to continue connecting (yes/no)? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are
prompted now it is to install the new keys
root@192.168.10.10's password:此处输入远程服务器密码
Number of key(s) added: 1
Now try logging into the machine, with: "ssh '192.168.10.10'"
and check to make sure that only the key(s) you wanted were added.
```

第 3 步：对服务器进行设置，使其只允许密钥验证，拒绝传统的口令验证方式。记得在修改配置文件后保存并重启 sshd 服务程序。

```
[root@linuxprobe ~]# vim /etc/ssh/sshd_config
.....省略部分输出信息.....
74
75 # To disable tunneled clear text passwords, change to no here!
76 #PasswordAuthentication yes
77 #PermitEmptyPasswords no
78 PasswordAuthentication no
79
.....省略部分输出信息.....
[root@linuxprobe ~]# systemctl restart sshd
```

第 4 步：在客户端尝试登录到服务器，此时无须输入密码也可成功登录。

```
[root@linuxprobe ~]# ssh 192.168.10.10
Last login: Mon Apr 13 19:34:13 2017
```

9.2.3 远程传输命令

scp (secure copy) 是一个基于 SSH 协议在网络之间进行安全传输的命令，其格式为“scp [参数] 本地文件 远程帐户@远程 IP 地址:远程目录”。

与第 2 章讲解的 cp 命令不同，cp 命令只能在本地硬盘中进行文件复制，而 scp 不仅能够通过网络传送数据，而且所有的数据都将进行加密处理。例如，如果想把一些文

件通过网络从一台主机传递到其他主机，这两台主机又恰巧是 Linux 系统，这时使用 scp 命令就可以轻松完成文件的传递了。scp 命令中可用的参数以及作用如表 9-2 所示。

表 9-2 scp 命令中可用的参数及作用

参数	作用
-v	显示详细的连接进度
-P	指定远程主机的 sshd 端口号
-r	用于传送文件夹
-6	使用 IPv6 协议

在使用 scp 命令把文件从本地复制到远程主机时，首先需要以绝对路径的形式写清本地文件的存放位置。如果要传送整个文件夹内的所有数据，还需要额外添加参数-r 进行递归操作。然后写上要传送到的远程主机的 IP 地址，远程服务器便会要求进行身份验证了。当前用户名称为 root，而密码则为远程服务器的密码。如果想使用指定用户的身份进行验证，可使用用户名@主机地址的参数格式。最后需要在远程主机的 IP 地址后面添加冒号，并在后面写上要传送到远程主机的哪个文件夹中。只要参数正确并且成功验证了用户身份，即可开始传送工作。由于 scp 命令是基于 SSH 协议进行文件传送的，而 9.2.2 小节又设置好了密钥验证，因此当前在传输文件时，并不需要账户和密码。

```
[root@linuxprobe ~]# echo "Welcome to LinuxProbe.Com" > readme.txt
[root@linuxprobe ~]# scp /root/readme.txt 192.168.10.20:/home
root@192.168.10.20's password:此处输入远程服务器中 root 管理员的密码
readme.txt 100% 26 0.0KB/s 00:00
```

此外，还可以使用 scp 命令把远程主机上的文件下载到本地主机，其命令格式为“scp[参数] 远程用户@远程 IP 地址:远程文件 本地目录”。例如，可以把远程主机的系统版本信息文件下载过来，这样就无须先登录远程主机，再进行文件传送了，也就省去了很多周折。

```
[root@linuxprobe ~]# scp 192.168.10.20:/etc/redhat-release /root
root@192.168.10.20's password:此处输入远程服务器中 root 管理员的密码
redhat-release 100% 52 0.1KB/s 00:00
[root@linuxprobe ~]# cat redhat-release
Red Hat Enterprise Linux Server release 7.0 (Maipo)
```

9.3 不间断会话服务

大家在学习 sshd 服务时，不知有没有注意到这样一个事情：当与远程主机的会话被关闭时，在远程主机上运行的命令也随之被中断。

如果我们正在使用命令来打包文件，或者正在使用脚本安装某个服务程序，中途是绝对不能关闭在本地打开的终端窗口或断开网络链接的，甚至是网速的波动都有可能導致任务中断，此时只能重新进行远程链接并重新开始任务。还有些时候，我们正在执行文件打包操作，同时又想用脚本来安装某个服务程序，这时会因为打包操作的输出信息占满用户的屏幕界面，而只能再打开一个执行远程会话的终端窗口，时间久了，难免会忘记这些打开的终端窗口是做什么用的了。

screen 是一款能够实现多窗口远程控制的开源服务程序，简单来说就是为了解决网络异常中断或为了同时控制多个远程终端窗口而设计的程序。用户还可以使用 **screen** 服务程序同时在多个远程会话中自由切换，能够做到实现如下功能。

- **会话恢复**：即便网络中断，也可让会话随时恢复，确保用户不会失去对远程会话的控制。
- **多窗口**：每个会话都是独立运行的，拥有各自独立的输入输出终端窗口，终端窗口内显示过的信息也将被分开隔离保存，以便下次使用时依然能看到之前的操作记录。
- **会话共享**：当多个用户同时登录到远程服务器时，便可以使用会话共享功能让用户之间的输入输出信息共享。

在 RHEL 7 系统中，没有默认安装 **screen** 服务程序，因此需要配置 Yum 仓库来安装它。首先将虚拟机的 CD/DVD 光盘选项设置为“使用 ISO 镜像文件”，并选择已经下载好的系统镜像，如图 9-12 所示。

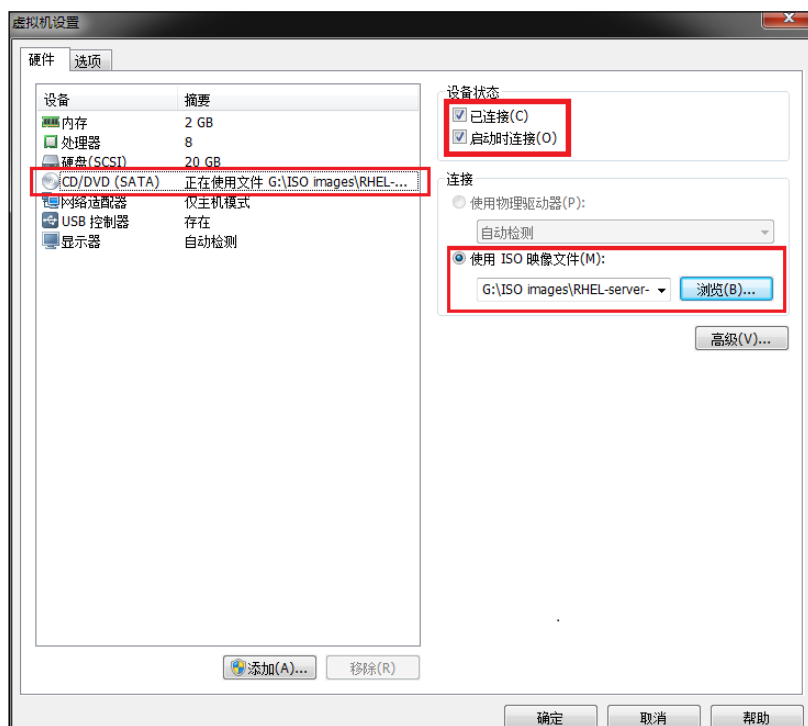


图 9-12 将虚拟机的光盘设备指向 ISO 镜像

然后，把光盘设备中的系统镜像挂载到 `/media/cdrom` 目录。

```
[root@linuxprobe ~]# mkdir -p /media/cdrom
[root@linuxprobe ~]# mount /dev/cdrom /media/cdrom
mount: /dev/sr0 is write-protected, mounting read-only
```

最后，使用 **Vim** 文本编辑器创建 Yum 仓库的配置文件。下述命令中用到的具体参数的含

义，可参考 4.1.4 小节。

```
[root@linuxprobe ~]# vim /etc/yum.repos.d/rhel7.repo
[rhel7]
name=rhel7
baseurl=file:///media/cdrom
enabled=1
gpgcheck=0
```

现在，就可以使用 Yum 仓库来安装 screen 服务程序了。简捷起见，刘遄老师将对后面章节中出现的 Yum 软件安装信息进行过滤——把重复性高及无意义的非必要信息省略。

```
[root@linuxprobe ~]# yum install screen
Loaded plugins: langpacks, product-id, subscription-manager
This system is not registered to Red Hat Subscription Management. You can use
subscription-manager to register.
rhel | 4.1 kB 00:00
Resolving Dependencies
--> Running transaction check
---> Package screen.x86_64 0:4.1.0-0.19.20120314git3c2946.el7 will be installed
--> Finished Dependency Resolution
Dependencies Resolved

=====
Package Arch Version Repository
Size
=====
Installing:
 screen x86_64 4.1.0-0.19.20120314git3c2946.el7 rhel 551 k
Transaction Summary
=====
Install 1 Package
Total download size: 551 k
Installed size: 914 k
Is this ok [y/d/N]: y
Downloading packages:
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Installing : screen-4.1.0-0.19.20120314git3c2946.el7.x86_64 1/1
  Verifying : screen-4.1.0-0.19.20120314git3c2946.el7.x86_64 1/1
Installed:
 screen.x86_64 0:4.1.0-0.19.20120314git3c2946.el7
Complete!
```

9.3.1 管理远程会话

screen 命令能做的事情非常多：可以用 -S 参数创建会话窗口；用 -d 参数将指定会话进行离线处理；用 -r 参数恢复指定会话；用 -x 参数一次性恢复所有的会话；用 -ls 参数显示当前已有的会话；以及用 -wipe 参数把目前无法使用的会话删除，等等。

下面创建一个名称为 backup 的会话窗口。请各位读者留心观察，当在命令行中敲下这条命令的一瞬间，屏幕会快速闪动一下，这时就已经进入 screen 服务会话中了，在里面运行的

任何操作都会被后台记录下来。

```
[root@linuxprobe ~]# screen -S backup
[root@linuxprobe ~]#
```

执行命令后会立即返回一个提示符。虽然看起来与刚才没有不同，但实际上可以查看到当前的会话正在工作中。

```
[root@linuxprobe ~]# screen -ls
There is a screen on:
32230.backup (Attached)
1 Socket in /var/run/screen/S-root.
```

要想退出一个会话也十分简单，只需在命令行中执行 `exit` 命令即可。

```
[root@linuxprobe ~]# exit
[screen is terminating]
```

在日常的生产环境中，其实并不是必须先创建会话，然后再开始工作。可以直接使用 `screen` 命令执行要运行的命令，这样在命令中的一切操作也都会被记录下来，当命令执行结束后 `screen` 会话也会自动结束。

```
[root@linuxprobe ~]# screen vim memo.txt
welcome to linuxprobe.com
```

为了演示 `screen` 不间断会话服务的强大之处，我们先来创建一个名为 `linux` 的会话，然后强行把窗口关闭掉（这与进行远程连接时突然断网具有相同的效果）：

```
[root@linuxprobe ~]# screen -S linux
[root@linuxprobe ~]#
[root@linuxprobe ~]# tail -f /var/log/messages
Feb 20 11:20:01 localhost systemd: Starting Session 2 of user root.
Feb 20 11:20:01 localhost systemd: Started Session 2 of user root.
Feb 20 11:21:19 localhost dbus-daemon: dbus[1124]: [system] Activating service
name='com.redhat.SubscriptionManager' (using servicehelper)
Feb 20 11:21:19 localhost dbus[1124]: [system] Activating service name='com.
redhat.SubscriptionManager' (using servicehelper)
Feb 20 11:21:19 localhost dbus-daemon: dbus[1124]: [system] Successfully activated
service 'com.redhat.SubscriptionManager'
Feb 20 11:21:19 localhost dbus[1124]: [system] Successfully activated service 'com.
redhat.SubscriptionManager'
Feb 20 11:30:01 localhost systemd: Starting Session 3 of user root.
Feb 20 11:30:01 localhost systemd: Started Session 3 of user root.
Feb 20 11:30:43 localhost systemd: Starting Cleanup of Temporary Directories...
Feb 20 11:30:43 localhost systemd: Started Cleanup of Temporary Directories.
```

由于刚才关闭了会话窗口，这样的操作在传统的远程控制中一定会导致正在运行的命令也突然终止，但在 `screen` 不间断会话服务中则不会这样。我们只需查看一下刚刚离线的会话名称，然后尝试恢复回来就可以继续工作了：

```
[root@linuxprobe ~]# screen -ls
There is a screen on:
13469.linux (Detached)
1 Socket in /var/run/screen/S-root.
[root@linuxprobe ~]# screen -r linux
```

```
[root@linuxprobe ~]#
[root@linuxprobe ~]# tail -f /var/log/messages
Feb 20 11:20:01 localhost systemd: Starting Session 2 of user root.
Feb 20 11:20:01 localhost systemd: Started Session 2 of user root.
Feb 20 11:21:19 localhost dbus-daemon: dbus[1124]: [system] Activating service
name='com.redhat.SubscriptionManager' (using servicehelper)
Feb 20 11:21:19 localhost dbus[1124]: [system] Activating service name='com.
redhat.SubscriptionManager' (using servicehelper)
Feb 20 11:21:19 localhost dbus-daemon: dbus[1124]: [system] Successfully
activated service 'com.redhat.SubscriptionManager'
Feb 20 11:21:19 localhost dbus[1124]: [system] Successfully activated service
'com.redhat.SubscriptionManager'
Feb 20 11:30:01 localhost systemd: Starting Session 3 of user root.
Feb 20 11:30:01 localhost systemd: Started Session 3 of user root.
Feb 20 11:30:43 localhost systemd: Starting Cleanup of Temporary Directories...
Feb 20 11:30:43 localhost systemd: Started Cleanup of Temporary Directories.
Feb 20 11:40:01 localhost systemd: Starting Session 4 of user root.
Feb 20 11:40:01 localhost systemd: Started Session 4 of user root.
```

如果我们突然又想到了还有其他事情需要处理，也可以多创建几个会话窗口放在一起使用。如果这段时间内不再使用某个会话窗口，可以把它设置为临时断开（detach）模式，随后在需要时再重新连接（attach）回来即可。这段时间内，在会话窗口内运行的程序会继续执行。

9.3.2 会话共享功能

screen 命令不仅可以确保用户在极端情况下也不丢失对系统的远程控制，保证了生产环境中远程工作的不间断性，而且它还具有会话共享、分屏切割、会话锁定等实用的功能。其中，会话共享功能是一件很酷的事情，当多个用户同时控制主机的时候，它可以把屏幕内容共享出来，也就是说每个用户都可以看到相同的内容。

screen 的会话共享功能的流程拓扑如图 9-13 所示。

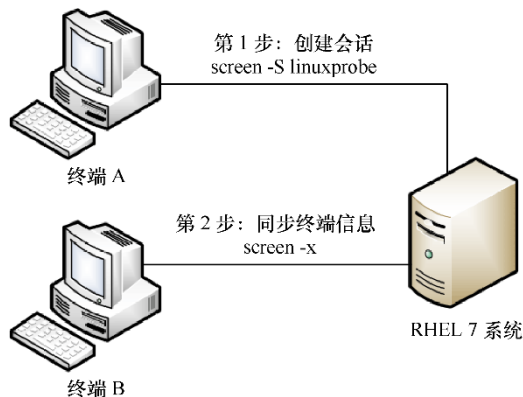


图 9-13 会话共享功能的流程拓扑

要实现会话共享功能，首先使用 ssh 服务程序将终端 A 远程连接到服务器，并创建一个会话窗口。

```
[root@client A ~]# ssh 192.168.10.10
The authenticity of host '192.168.10.10 (192.168.10.10)' can't be established.
ECDSA key fingerprint is 70:3b:5d:37:96:7b:2e:a5:28:0d:7e:dc:47:6a:fe:5c.
Are you sure you want to continue connecting (yes/no)? yes
```

```
Warning: Permanently added '192.168.10.10' (ECDSA) to the list of known hosts.
root@192.168.10.10's password: 此处输入 root 管理员密码
Last login: Wed May 4 07:56:29 2017
[root@client A ~]# screen -S linuxprobe
[root@client A ~]#
```

然后，使用 ssh 服务程序将终端 B 远程连接到服务器，并执行获取远程会话的命令。接下来，两台主机就能看到相同的内容了。

```
[root@client B ~]# ssh 192.168.10.10
The authenticity of host '192.168.10.10 (192.168.10.10)' can't be established.
ECDSA key fingerprint is 70:3b:5d:37:96:7b:2e:a5:28:0d:7e:dc:47:6a:fe:5c.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.10.10' (ECDSA) to the list of known hosts.
root@192.168.10.10's password: 此处输入 root 管理员密码
Last login: Wed Feb 22 04:55:38 2017 from 192.168.10.10
[root@client B ~]# screen -x
[root@client B ~]
```

复习题

1. 在 Linux 系统中有多种方法可以配置网络参数，请列举几种。

答：配置网卡参数可以使用 nmtui 命令、nmcli 命令或者直接编辑网卡配置文件来实现对网卡参数的修改。

2. 在 RHEL 7 系统中使用网卡会话技术的目的是什么？

答：使用 nmcli 命令来管理网卡会话的目的是为了快速切换网卡参数，以便适应不同的工作场景。

3. 请简述网卡绑定技术 mode6 模式的特点。

答：平时两块网卡均工作，且自动备援，无须交换机设备提供辅助支持。

4. 在 Linux 系统中，当通过修改其配置文件中的参数来配置服务程序时，若想要让新配置参数生效，还需要执行什么操作？

答：需要重新启动相关的服务程序，或让服务程序重新加载配置文件，或重启系统。

5. sshd 服务的口令验证与密钥验证方式，哪个更安全？

答：一般情况下，密钥验证方式更加安全。若用户若认证有更高的安全需求，还可以再对密钥文件进行口令加密，从而实现双重加密。

6. 想要把本地文件/root/out.txt 传送到地址为 192.168.10.20 的远程主机的/home 目录下,且本地主机与远程主机均为 Linux 系统,最为简便的传送方式是什么?

答: 执行命令 `scp /root/out.txt root@192.168.10.20:/home`, 并在进行口令验证后即可开始传送。

7. 请简述配置 Yum 仓库的步骤。

答: 首先应该创建挂载目录并把光盘镜像文件与其关联, 然后修改 Yum 的配置文件, 填入相关参数, 尤其需要注意挂载目录的存放路径要正确无误, 最后便可使用 Yum 命令来安装相关的服务程序了。

8. screen 服务程序能够让用户实现远程控制的不间断会话服务, 即便网络发生中断也不丢失对远程主机的会话控制。那么, 当想要恢复到一个名为 linux 的会话窗口时, 应该怎么做呢?

答: 执行命令 `screen -r linux` 即可恢复到这个会话窗口中。